Will They Make The Shot?

Dawson Tree & Ethan Bates
CS 5830

Introduction

To win a basketball game, a team obviously needs to make shots. The aim of this project is to provide useful information that could be used by a professional basketball team to improve their teams' shooting performance. To achieve this purpose, we analyzed two different datasets, one with basic in-game NBA shot information, and another with detailed biomechanical data. Both logistic regression and support vector machines were used to classify whether shots were made or missed. Some final recommendations to help teams shoot better in games include taking less jump shots late in games and avoiding taking other types of shots from the center of the court.

Presentation Slides: Project 7

GitHub Repo:https://github.com/Grumblehead84/CS5830_Project7

Dataset 1: https://www.kaggle.com/datasets/mexwell/nba-shots/code

The first dataset we used contains data from NBA games played from 2004-2024. This dataset contains data for every shot that would be very easily accessible by any professional team, such as shot location, shot type, and the timing of the shot during the game. The primary goal of a classifier trained on this dataset would be to help teams know what shot attributes affect whether a shot is made or not and how. Because this data is readily accessible to teams, they can instantly implement these findings into their game strategy. As for cleaning the data, we got rid of shots taken in quadruple overtime because there weren't enough taken to truly determine the effect the specific time period has on shots. For later modeling purposes, a new variable, jump shot or non-jump shot, was also created.

Analysis Technique

The goal of the analysis of the NBA dataset was to determine the effect of shot characteristics on the probability of each shot being made and give easily interpretable results that could be implemented by an NBA team. Logistic regression was used to achieve this goal because the response variable, whether a shot is made or not, is binary and its results would be easy to interpret and explain to stakeholders. Our final logistic regression model does use an interaction term for every predictor, capturing the effect of each predictor separately for jump shots and non-jump shots such as dunks and layups. This approach shows some distinct differences between the two types of shots and explains more of the variation in the data.

Results

The final model that performed best was a logistic regression model using the following variables, a binary variable indicating whether a shot is a jump shot, a categorical variable indicating the location of the shot (right, left, or center), a categorical variable indicating the quarter the shot was taken, the standardized s\hot distance, and the standardized time left in

the quarter. Like previously stated, this model used interaction terms to calculate these effects separately for jump shots and non-jump shots.

OLS Regression Results

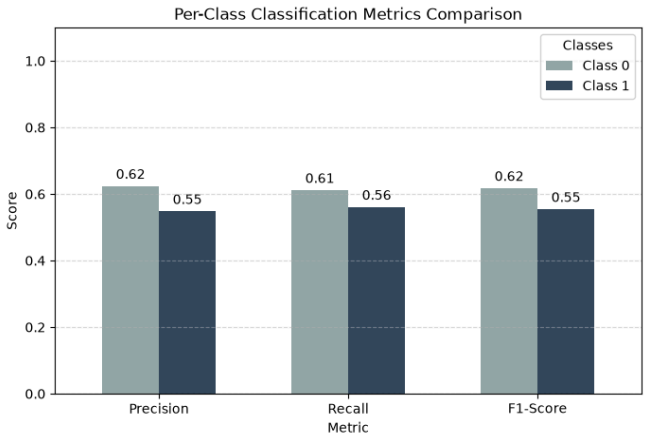
Dep. Variable:	shot_made	R-squared:	0.055
Model:	OLS	Adj. R-squared:	0.055
Method:	Least Squares	F-statistic:	1.105e+04
Date:	Mon, 09 Nov 2026	Prob (F-statistic):	0.00
Time:	07:08:47	Log-Likelihood:	-2.7902e+06
No. Observations:	4019629	AIC:	5.580e+06
Df Residuals:	4019607	BIC:	5.581e+06
Df Model:	21		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.4022	0.001	584.945	0.000	0.401	0.404
non_jump_shot[T.True]	-0.1101	0.002	-47.801	0.000	-0.115	-0.106
right[T.True]	0.0009	0.001	1.150	0.250	-0.001	0.002
left[T.True]	0.0015	0.001	1.962	0.050	1.4e-06	0.003
quarter_2[T.True]	-0.0114	0.001	-13.483	0.000	-0.013	-0.010
quarter_3[T.True]	-0.0066	0.001	-7.813	0.000	-0.008	-0.005
quarter_4[T.True]	-0.0230	0.001	-27.005	0.000	-0.025	-0.021
quarter_5[T.True]	-0.0343	0.004	-8.920	0.000	-0.042	-0.027
quarter_6[T.True]	-0.0453	0.010	-4.469	0.000	-0.065	-0.025
quarter_7[T.True]	-0.0379	0.026	-1.479	0.139	-0.088	0.012
non_jump_shot[T.True]:right[T.True]	0.0557	0.003	18.041	0.000	0.050	0.062
non_jump_shot[T.True]:left[T.True]	0.0459	0.003	15.717	0.000	0.040	0.052
non_jump_shot[T.True]:quarter_2[T.True]	0.0084	0.001	5.929	0.000	0.006	0.011
non_jump_shot[T.True]:quarter_3[T.True]	-0.0028	0.001	-1.998	0.046	-0.006	-5.38e-05
non_jump_shot[T.True]:quarter_4[T.True]	0.0130	0.001	9.029	0.000	0.010	0.016
non_jump_shot[T.True]:quarter_5[T.True]	0.0002	0.007	0.034	0.973	-0.013	0.014
non_jump_shot[T.True]:quarter_6[T.True]	0.0141	0.018	0.786	0.432	-0.021	0.049
non_jump_shot[T.True]:quarter_7[T.True]	-0.0426	0.044	-0.960	0.337	-0.130	0.044
shot_distance_std	-0.0237	0.000	-57.488	0.000	-0.024	-0.023
non_jump_shot[T.True]:shot_distance_std	-0.2685	0.002	-140.935	0.000	-0.272	-0.265
total_secs_left_std	0.0121	0.000	40.443	0.000	0.012	0.013
non_jump_shot[T.True]:total_secs_left_std	-0.0125	0.001	-24.419	0.000	-0.013	-0.011

Omnibus:	16551737.440	Durbin-Watson:	2.000
Prob(Omnibus):	0.000	Jarque-Bera (JB):	528851.762
Skew:	0.176	Prob(JB):	0.00
Kurtosis:	1.258	Cond. No.	265.

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Model Results



Accuracy Metrics

Many factors related to a basketball shot are difficult to quantify, but using these basic statistics, we are able to explain 5.5% of the variation in the training set (95% of the data). Using validation data, this data reported an F1 score of 0.62 for predicting missed shots and 0.55 for predicting made shots. Some key takeaways from these results are that jump shot accuracy decreases over the length of a game, with a clear trend showing this both between and within quarters likely due to fatigue and higher accuracy required for these shots. Another is that non-jump shots taken from the right or left sections of the court have a 5% higher chance of being a made shot than shots from the center section. Both of these findings could be incorporated into in-game strategy by NBA teams.

Technical

To process this data, quadruple overtime shots were filtered out because there were too few to assess their effect. The exact location of each shot was turned into an angle from the baseline to classify a shot as being in the left, right, or center section. Shot types that included the word ‘Jump’ were defined as jump shots as well. Both of these categorical variables and the quarter variable were one-hot encoded for use in the logistic regression model. Continuous variables were also scaled. Because the dataset was so large, we were able to have a smaller validation set consisting of only 5% of the data that still contained 211560 shots. Logistic regression is appropriate for this dataset because the outcome is binary and the primary goal is interpretability. Using interaction terms is also appropriate because there are clear differences between a jump shot and other shots. In previous iterations of the model, there was no interaction term and we also tried to use the exact angle a shot was taken from as a predictor.

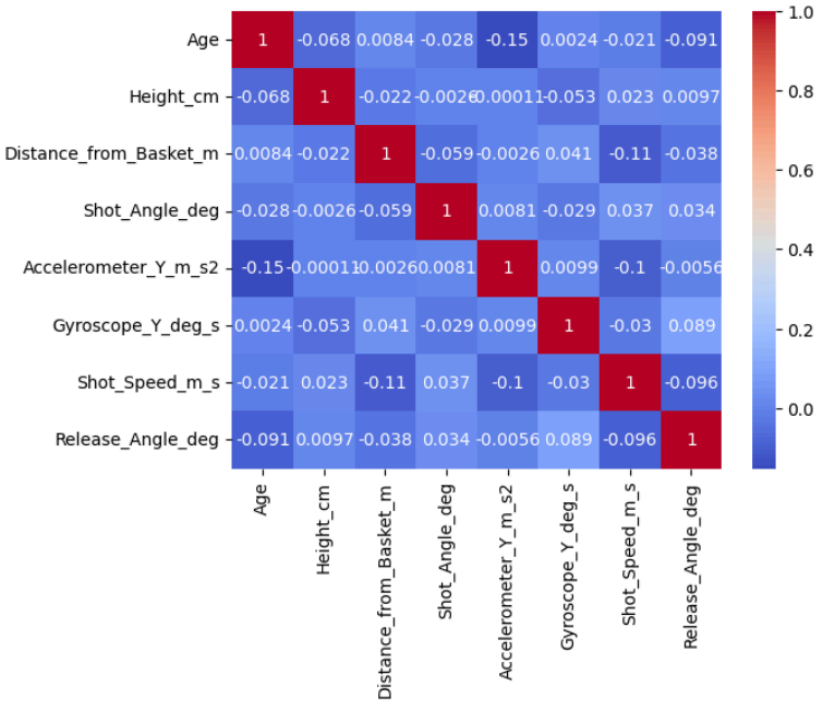
However, the final model proved to be more accurate and informative. We did try to fit an SVM to the data as well, but due to the size of the dataset and the computation required, we were unable to fit a linear model on a laptop computer without it crashing.

Dataset 2: Biomechanical Basketball Shooting Dataset

The second dataset we chose to look at focused more on the physical aspects of the shooters themselves. The data was collected using “IoT-based wristband[s] equipped with an MPU-6050 sensor,” and included 300 rows of observations. As players made different kinds of shots, physical data, including shot angles, acceleration of the shot within each 3d vector, shot speed, and distance from the basket were all measured. This data combined to create a comprehensive dataset about shots that was extremely conducive to logistic regression and support vector machines, as all this data was collected and connected to whether or not the shot was actually made. The data was gathered in 2024 and was already cleaned, with 0 Null values and standardized column data.

Analysis Technique

For this dataset, we used both logistic regression (OLS for better statistic reporting) and support vector machines to analyze the data. We felt that logistic regression would work well for this because it has the perfect column to predict on, Shot_Result, and many non-correlated variables.



We also wanted to try support vector machines because the data included spatial data in 3 dimensions, so we felt that support vector machines would help to better train on more complex

data. We trained 4 separate models for both types to see which combinations of features would work best for analysis. The four models for each type consisted of:

1. All numeric variables (Excluded shot type in the analysis)
2. All numeric variables + shot type one-hot encoded
3. All numeric variables, but with Accelerometer and Gyroscope values combined into one feature
4. Hand-picked variables

Results

The results showed that both SVM and Logistic models worked about as well as a coin flip when it comes to predicting shots made based on biomechanical data. Interestingly, the best working support vector machine model was the model that only included hand-selected variable, and for logistic regression, the hand-selected model came behind only the all-numeric model. This indicates to us that possibly less features could be better for making these predictions, or that including more complex data causes the models to work worse.

Model Comparison: Performance on Shot Made (Class 1)

	Model Type	Configuration	Accuracy	Precision (Class 1)	Recall (Class 1)	F1-Score (Class 1)
0	Logistic Regression	LogReg 1 (All Numeric)	0.5333	0.5357	0.5000	0.5172
1	SVM	SVM 3 (Hand-Selected)	0.5500	0.5652	0.4333	0.4906
2	Logistic Regression	LogReg 3 (Hand-Selected)	0.5000	0.5000	0.4333	0.4643
3	SVM	SVM 1 (All Numeric)	0.5167	0.5217	0.4000	0.4528
4	SVM	SVM 4 (Sensor Magnitudes)	0.4500	0.4483	0.4333	0.4407
5	SVM	SVM 2 (Numeric + Shot_Type)	0.4667	0.4615	0.4000	0.4286
6	Logistic Regression	LogReg 4 (Sensor Magnitudes)	0.4667	0.4615	0.4000	0.4286
7	Logistic Regression	LogReg 2 (Numeric + Shot_Type)	0.4167	0.4194	0.4333	0.4262

We also found that results were poor whether or not polynomial SVM was used over RBF SVM. In fact, RBF performed the best out of the three!
RBF Accuracy: 0.5166666666666667
Poly 2 Accuracy: 0.48333333333333334
Poly 3 Accuracy: 0.45

Technical

To combine all accelerometer and gyroscopic values into one single vector value for new columns, we squared the values from each x, y, and z column, summed up the results, then squared the final result to derive the final magnitude. We also employed one-hot encoding for Shot_Type. Different datasets were pre-prepared for each model analysis and passed into pipelines for ease of training and testing for each model.